

Memory Management Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF
GOTHENBURG

What to read (main textbook)

- Sections 3.1, 3.2, and the introduction to Section 3.7 (stop before Section 3.7.1).
- Optional additional reading: Sections 8.1–8.5 of *Operating System Concepts* from Silberschatz.

Objective

1. Discuss how different types of memory are used
2. Discuss how information is continuously moved between “slow memory” and “fast memory” (accessed directly by the CPU)
3. Discuss why a process’ view of memory and physical memory are kept separated

Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

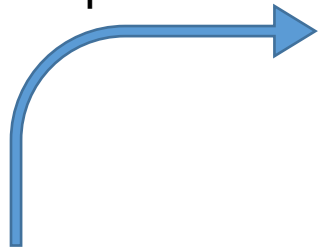
Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

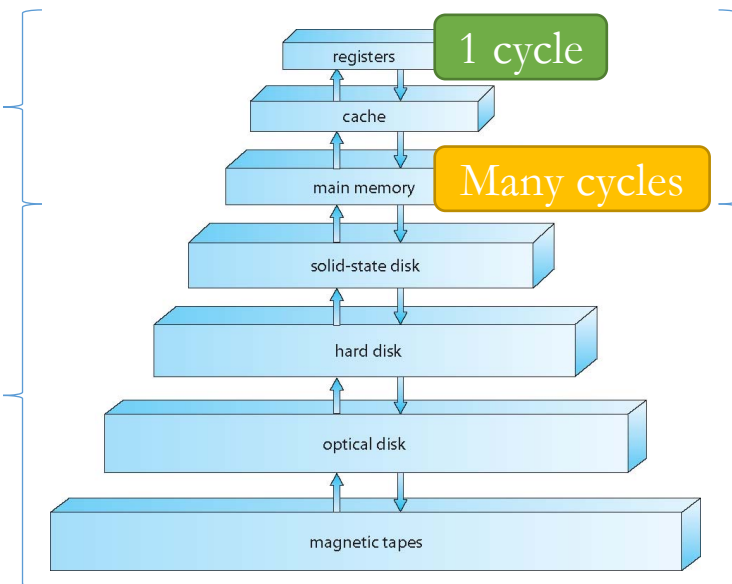
Background and basic hardware (I)

One goal of the OS is to keep the CPU busy by allowing multiple processes to share physical memory safely and efficiently

We first have to
move it up here!



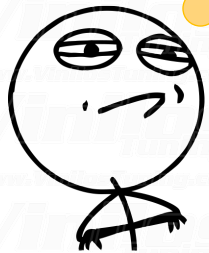
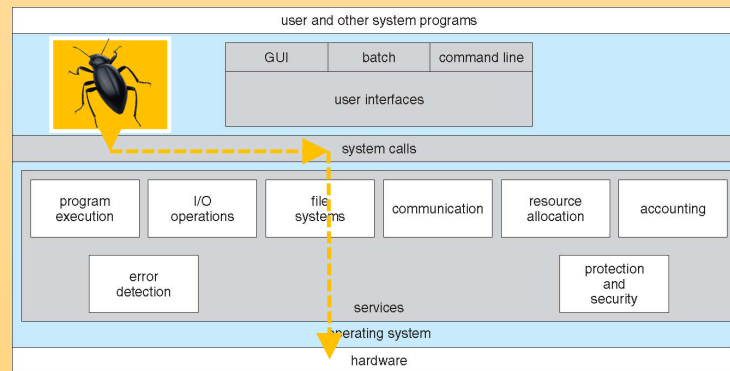
If some data the CPU
wants to access is here



Can be accessed
directly by the CPU

Cannot be accessed
directly by the CPU

Background and basic hardware (II)

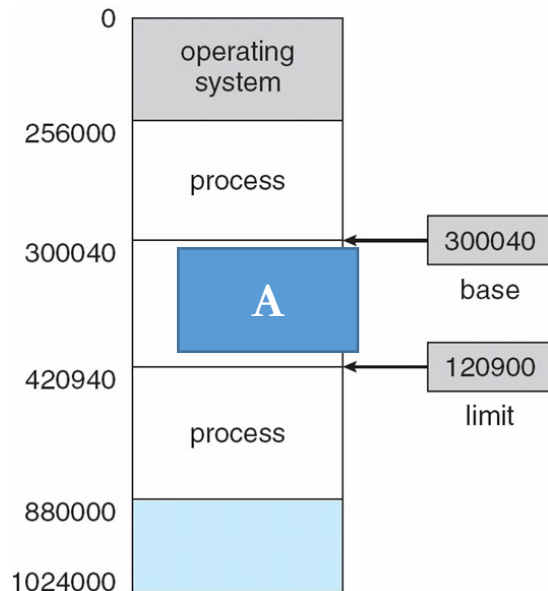


Need for fast CPU access to registers / caches and main memory?
→ The hardware performs ordinary register, cache, and memory accesses without invoking the OS for every access.
CHALLENGE: how does the OS configure the protection and address-translation mechanisms.?

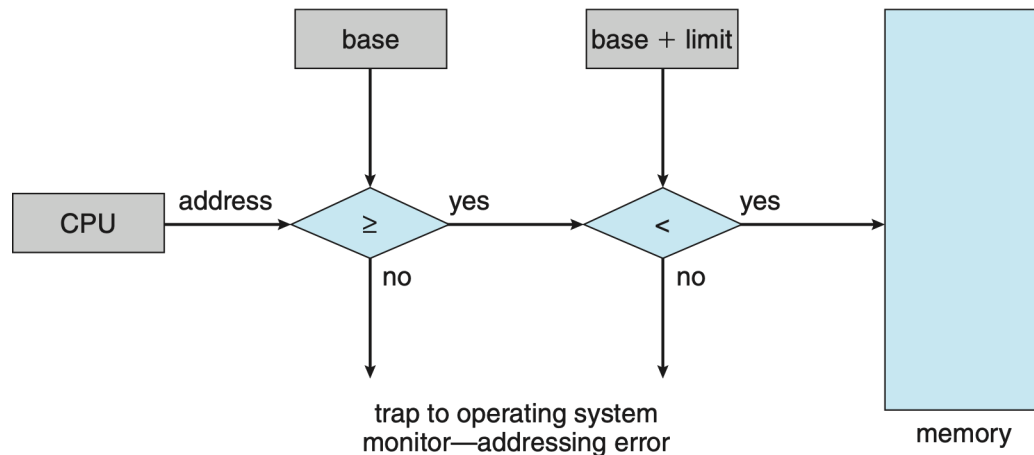
Agenda

- Introduction
- **Base and limit registers**
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

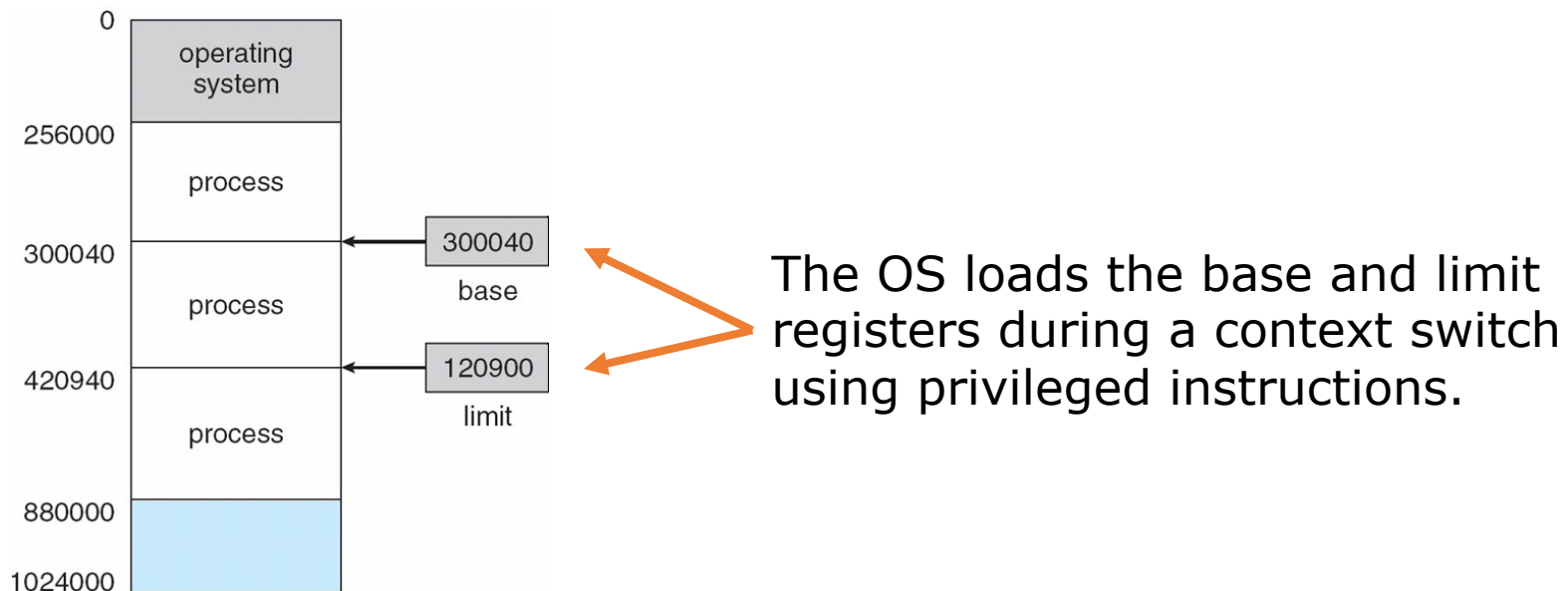
Base and Limit register (I)



- For each process, the CPU maintains two registers:
 - Base register: first address that can be accessed by the process
 - Limit register: number of addresses (starting from the base register) that can be accessed by the process
- The registers are loaded by the OS during a context switch
- Example: process A can only access addresses from 300040 to 420939



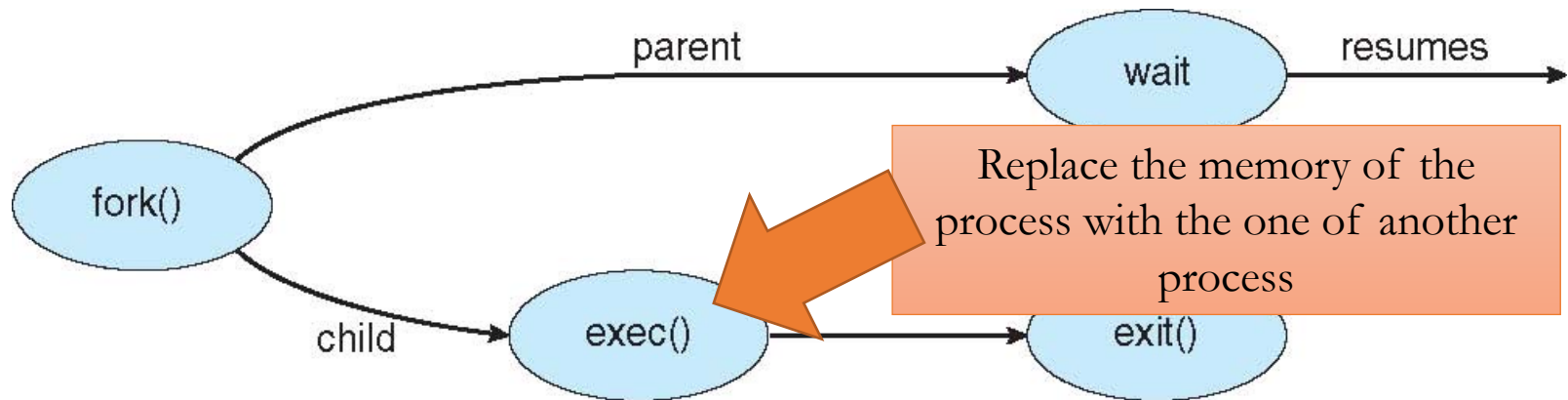
Base and Limit register (II)



Base and Limit register (III)

In this simplified base-and-limit model, kernel-mode code can access all physical memory.

A previous example is `exec()`: it replaces the current process image with a new program.



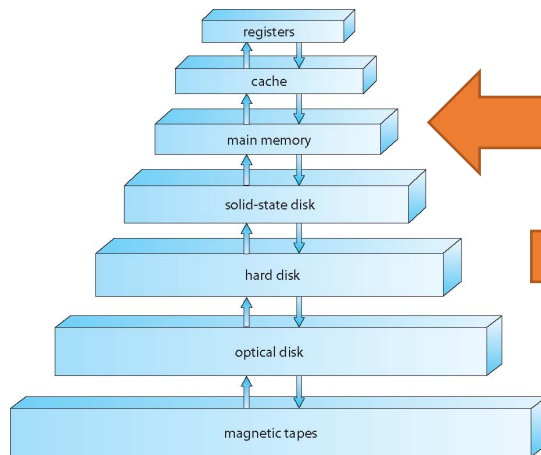
Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Address binding

...
 $a = b + 2$
...

← Where do we store a ?



← Once we load it into memory, we know the address

→ If the program is not loaded in memory, we do not know the address

Note: The compiler and linker assign logical addresses to program instructions and data. These logical addresses may be bound to physical addresses when the program is loaded or translated dynamically during execution by the MMU (this is when we use Virtual Memory, we discuss that in a dedicated lecture).

Logical/Virtual vs Physical address space (i)

Logical/Virtual address

...
a = b + 2
...

→ First address of the program 00000

→ Address of variable a 346

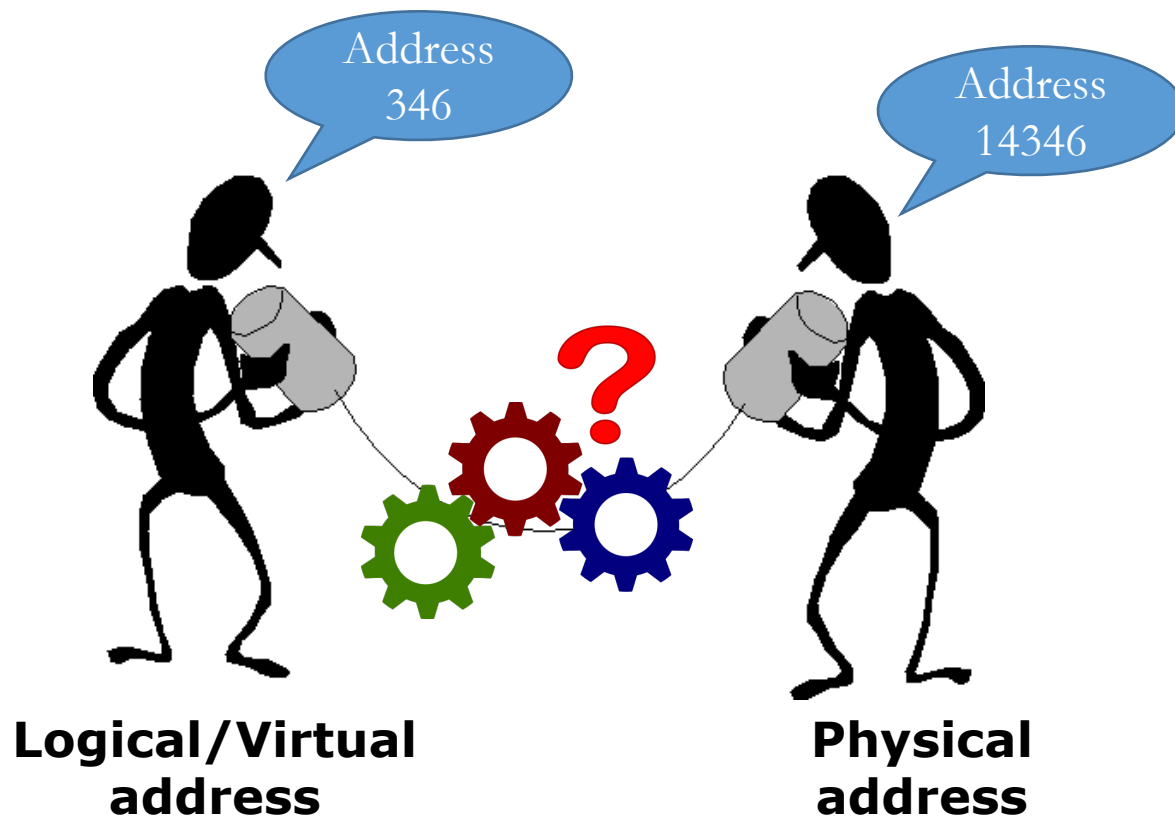
Physical address (main memory)

...
a = b + 2
...

→ First address of the program 14000

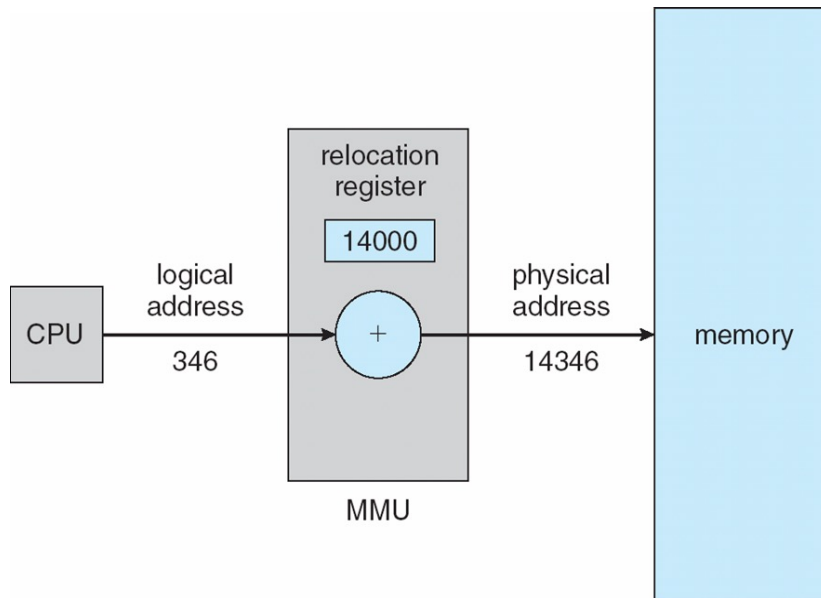
→ Address of variable a 14346

Logical/Virtual vs Physical address space (ii)



How are logical/virtual addresses converted into physical addresses?

Memory-Management Unit (MMU) – Relocation Register



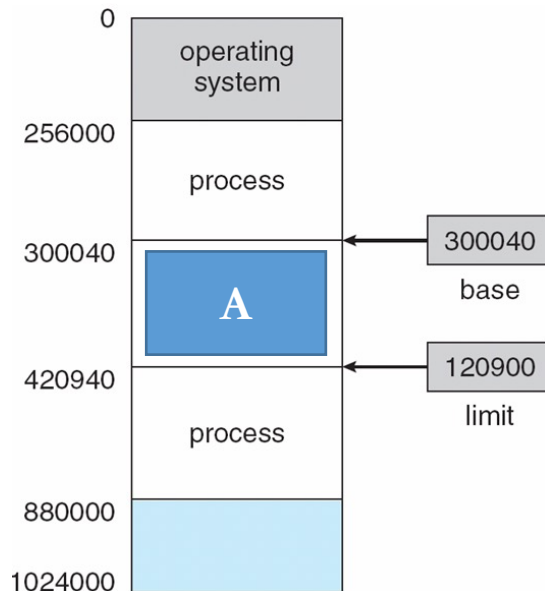
Logical/Virtual addresses $[0, \text{max}]$

Physical addresses $[R+0, R+\text{max}]$

Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- **Dynamic loading and dynamic linking [self-study]**
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Optimizations – Dynamic loading



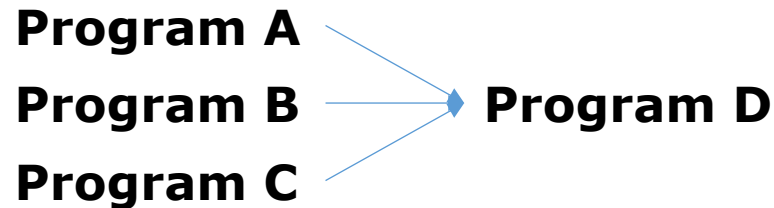
We assumed so far the entire program A is loaded

If program A is large but rarely uses some routines, loading the entire program wastes time and memory.

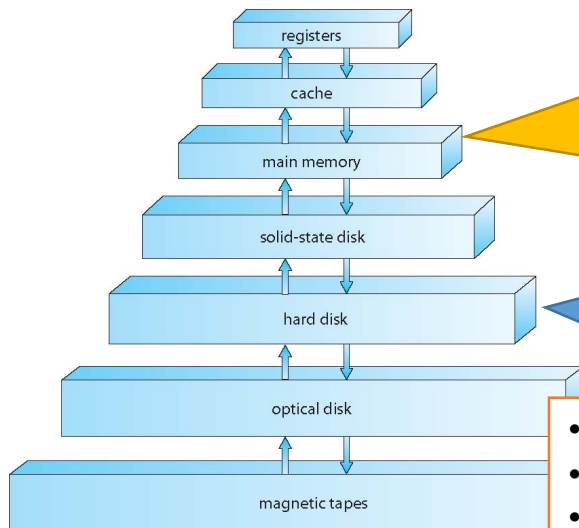
Dynamic loading loads a routine only when it is first needed.

Optimizations – Dynamic Linking (i)

Use library from



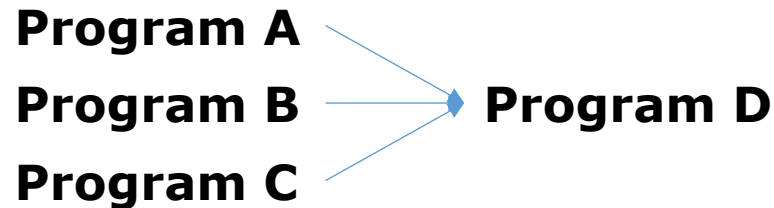
Static linking



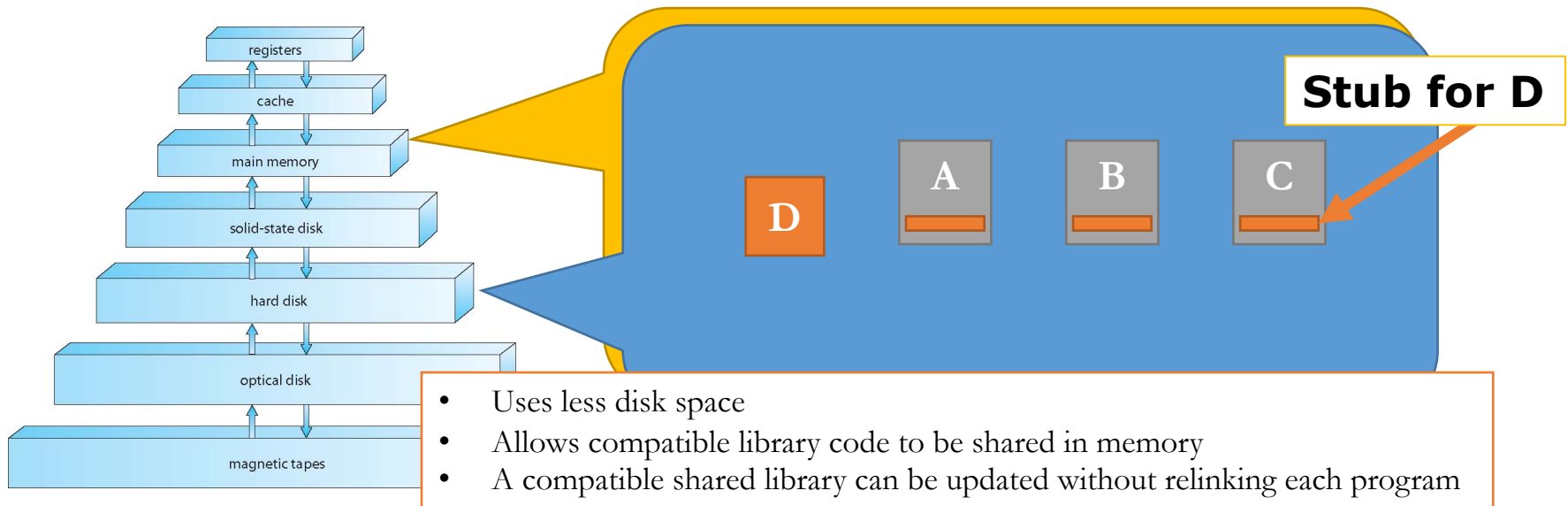
- Wastes disk space
- May create a separate in-memory copy of the library for each process
- Programs must be relinked when the library changes

Optimizations – Dynamic Linking (ii)

Use library from



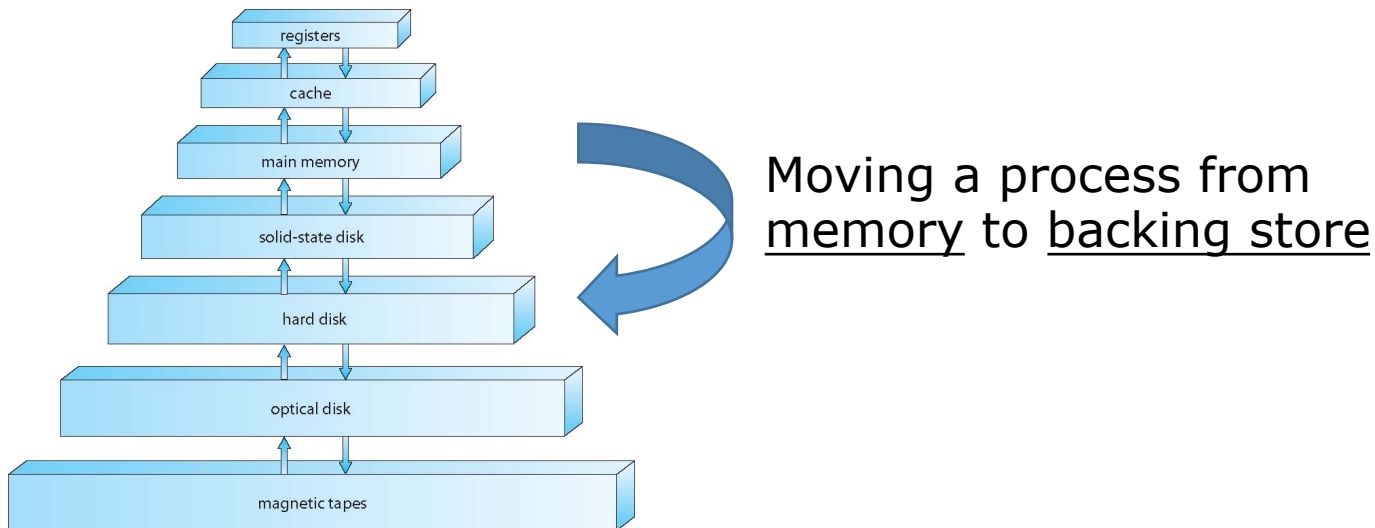
Dynamic linking



Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- **Swapping**
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Swapping (i)

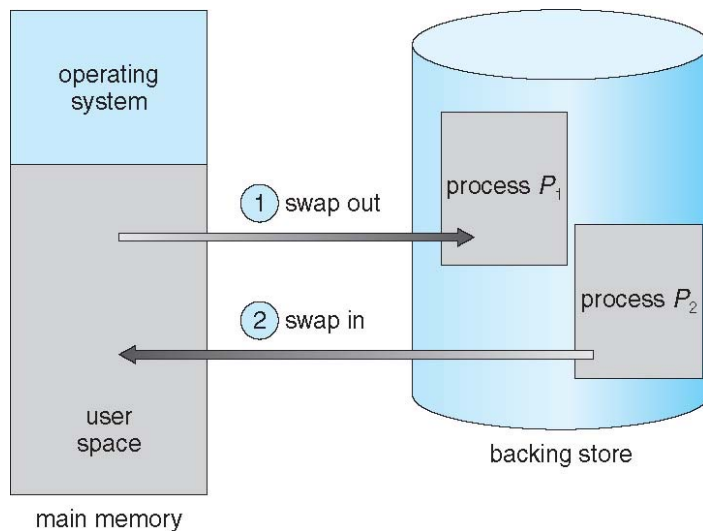


Why?

size(all logical addresses of all processes)
>
size(real physical memory)

Swapping (ii)

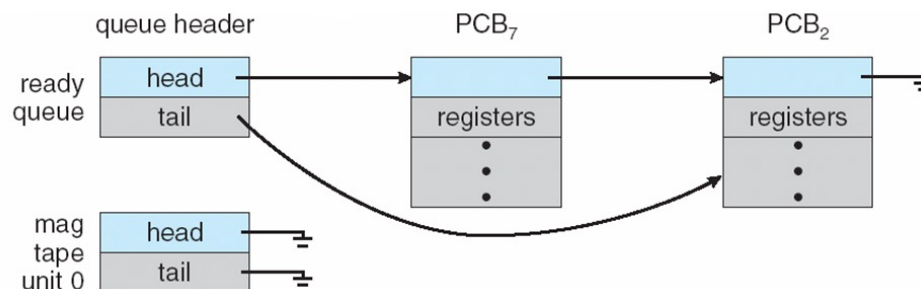
- Basic idea:



Questions:

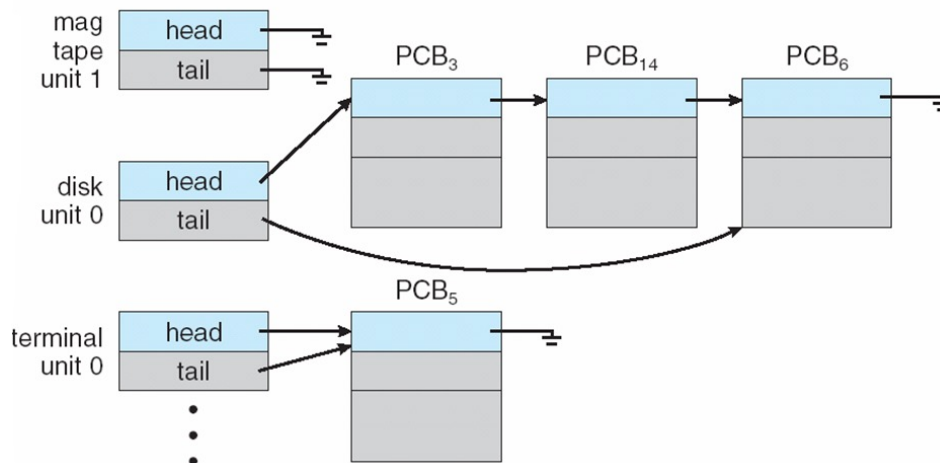
- Which processes can we swap?
- How much time does it take?
- When does it make sense to use swapping?

Swapping - Which processes can we swap?



Depends on the pending I/O...

Can the I/O results be buffered at the OS?



- A process with pending I/O cannot safely be swapped out if the device will transfer data directly into that process's memory.
- Possible solutions include waiting for the I/O to complete or using OS buffers.

Swapping – How much time does it take?

- Process in memory 100 MB
- Disk transfer rate 50 MB/s

→ $100 \text{ MB} / 50 \text{ MB/s} = 2 \text{ seconds}$

→ Swap out + Swap in = 4 seconds

(Notice: being optimistic! Disk writes/reads might be requested by other processes, interrupts will occur in the meantime, etc...)

It is slow and expensive (because write/read to disks are slower and more expensive than memory ones)!

Swapping – When does it make sense to use it?

- Not used “as it is”...
- Modified versions are used in UNIX, Linux and Windows
 - Swapping is activated when free memory goes below a certain threshold
 - Swapping is deactivated when free memory goes above a certain threshold
- Modified swapping (swap out / in only parts of the process) used in conjunction with Virtual Memory (next lecture...)

Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- **Memory allocation**
 - Contiguous allocation
 - Segmentation
 - Paging

Memory allocation

- Memory contains
 - The operating system itself
 - User processes
- Challenge: how to allocate memory efficiently?
- We will see 3 cases
 - Contiguous allocation
 - Segmentation
 - Paging



Logical and physical
memory more and
more separated

Less fragmentation (better
memory utilization)

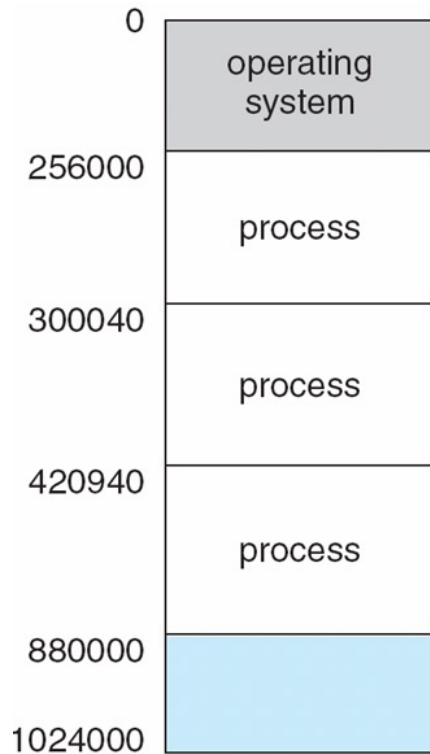
Protection

Faster loading
/ swapping

Agenda

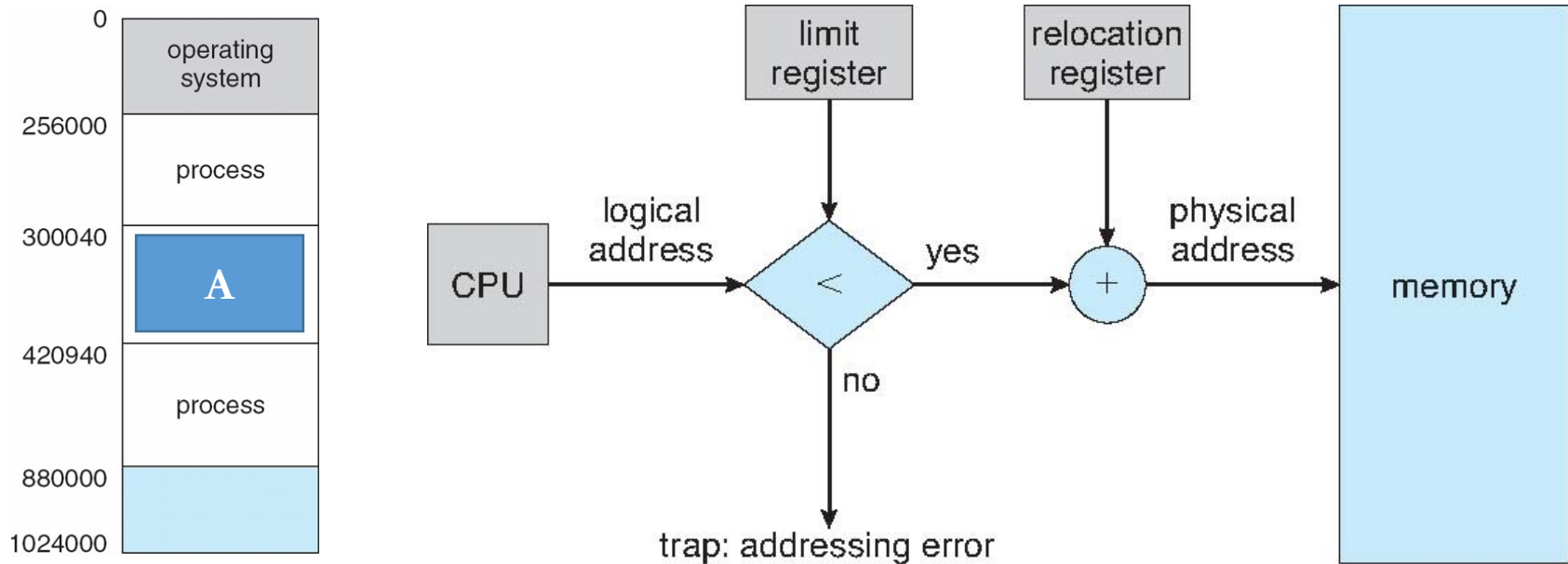
- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Contiguous allocation

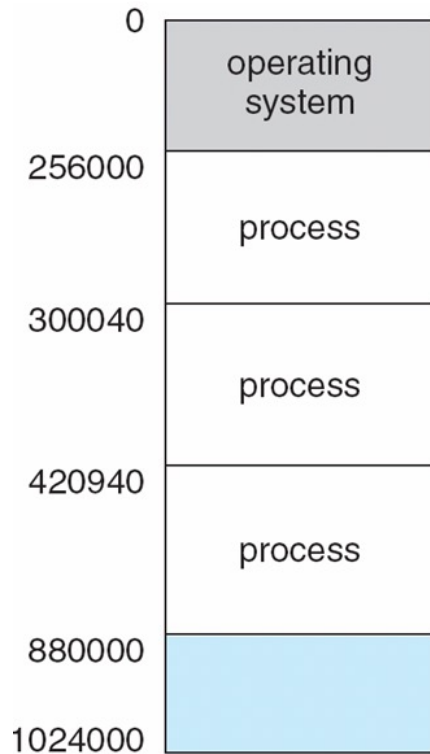


- The OS occupies a reserved region of physical memory, shown here at low addresses.
- Each process occupies one contiguous physical-memory region.
- Free holes may exist between processes.

Contiguous allocation – memory protection



Contiguous allocation - limitations



- When we terminate (or swap) a process
 - We create a **hole**
- The OS needs to keep track of holes to reuse them later on for other processes
- External fragmentation
 - sum of free-hole sizes $\geq$ requested allocation size
 - BUT
 - every individual hole $<$ requested allocation size

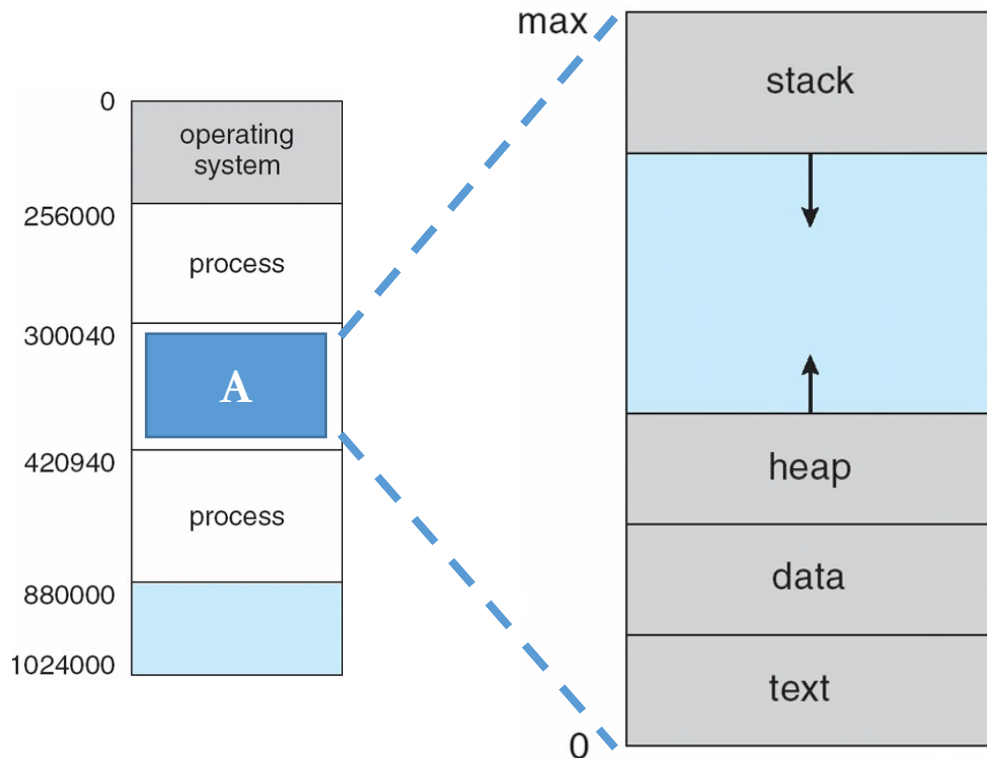
Contiguous allocation – which hole to assign to a new process?

- First fit → allocate the first hole that is big enough
- Best fit → allocate the smallest hole that is big enough
 - PROS: produces the smallest leftover hole
 - CONS: Search entire list (or keep list of holes sorted)
- Worst fit → allocate the largest hole
 - PROS: produces the largest leftover hole
 - CONS: Search entire list (or keep list of holes sorted)

Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

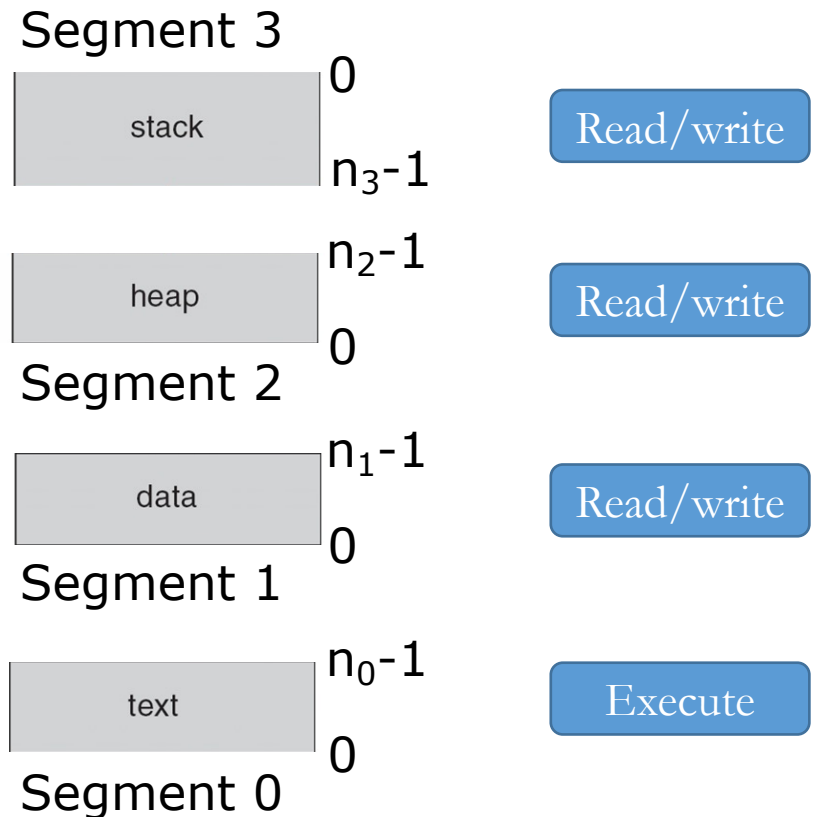
Segmentation (i)



- So far, we referred to the memory of a process as a single one-dimensional space $[0, \text{max}]$
- Nevertheless, from the programmer/program perspective:
 - Different parts have different meaning, and might need different access permissions
 - Different parts have different and variable size

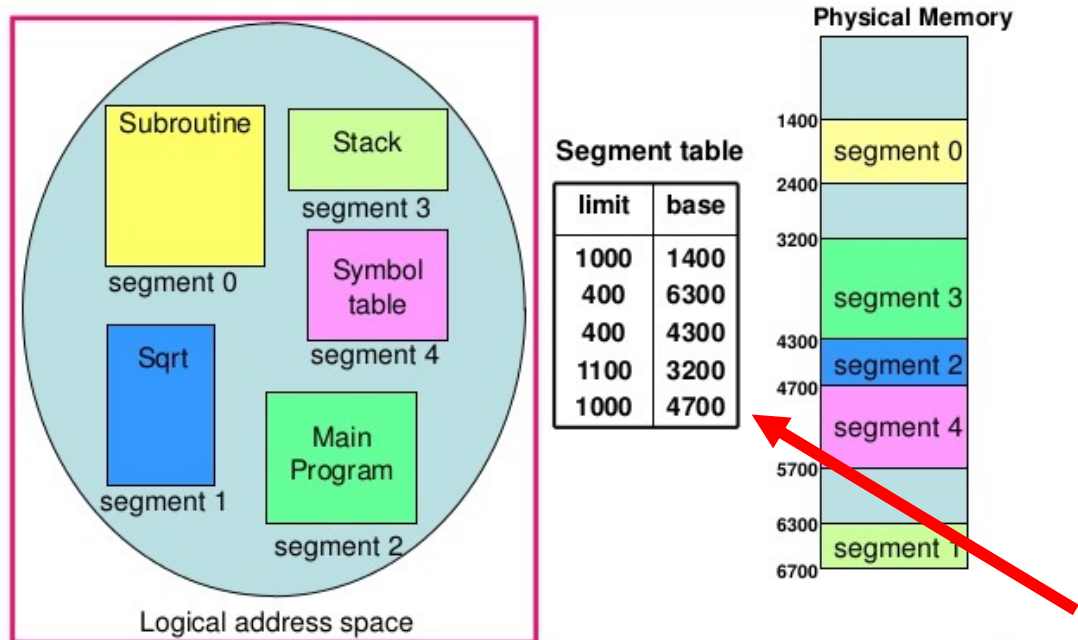
Segmentation (ii)

- With segmentation, a logical address is a pair $\langle \text{segment number}, \text{offset} \rangle$.



- The hardware verifies that the offset lies within the selected segment and that the requested access is permitted.
- Allows to have different permissions for the different segments!
- Different segments can occupy different physical-memory regions.
- Each segment must still occupy a contiguous physical region, so segmentation can suffer from external fragmentation.

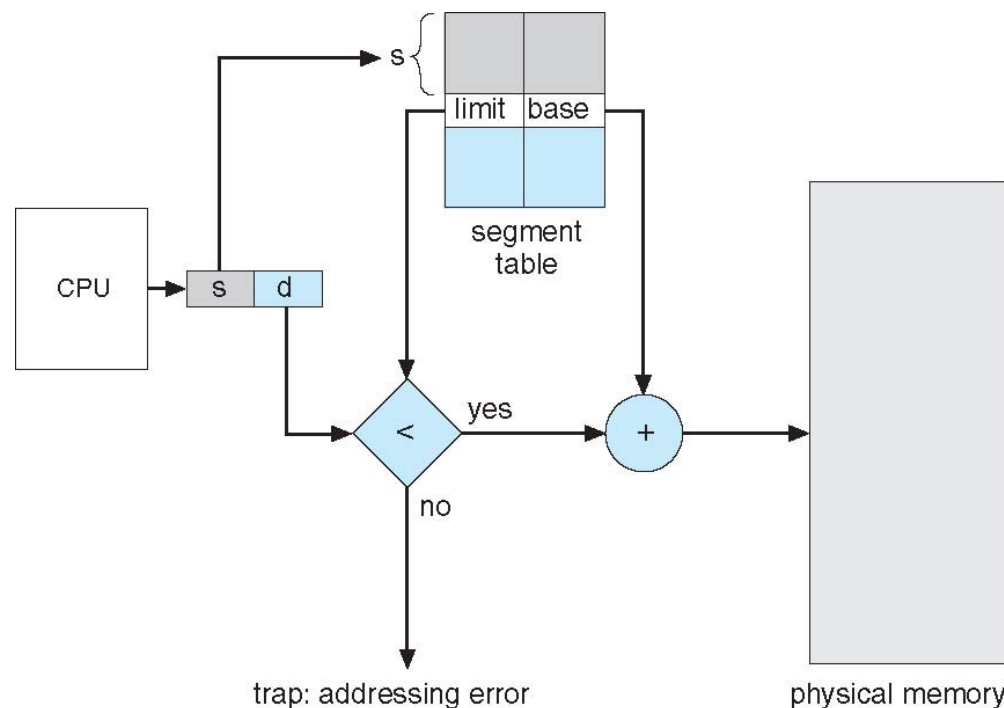
Segmentation – segment table



The segment table maps each logical segment number to a physical base address and a segment limit.

Segmentation – hardware support

The hardware can do this conversion automatically (and check whether the process is trying to access memory outside the segment)



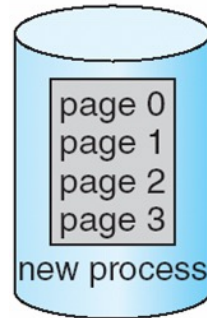
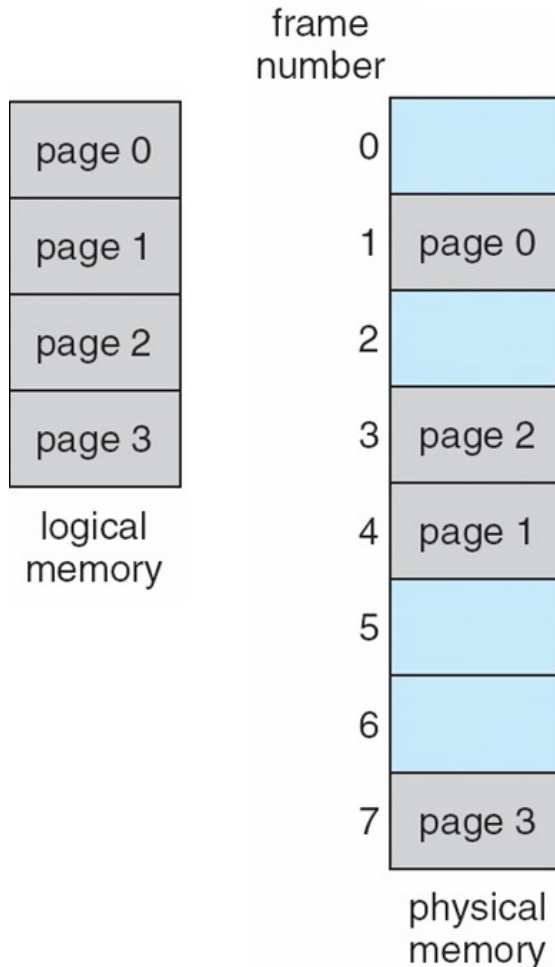
Agenda

- Introduction
- Base and limit registers
- Logical/virtual versus physical addresses
- Dynamic loading and dynamic linking [self-study]
- Swapping
- **Memory allocation**
 - Contiguous allocation
 - Segmentation
 - **Paging**

Paging

- Mechanism that allows for the physical address space used by a process to be non-contiguous
- Used in most modern OSes (from mainframes to smartphones)
- Requires cooperation between OS and hardware (as common)

Paging – Basic idea



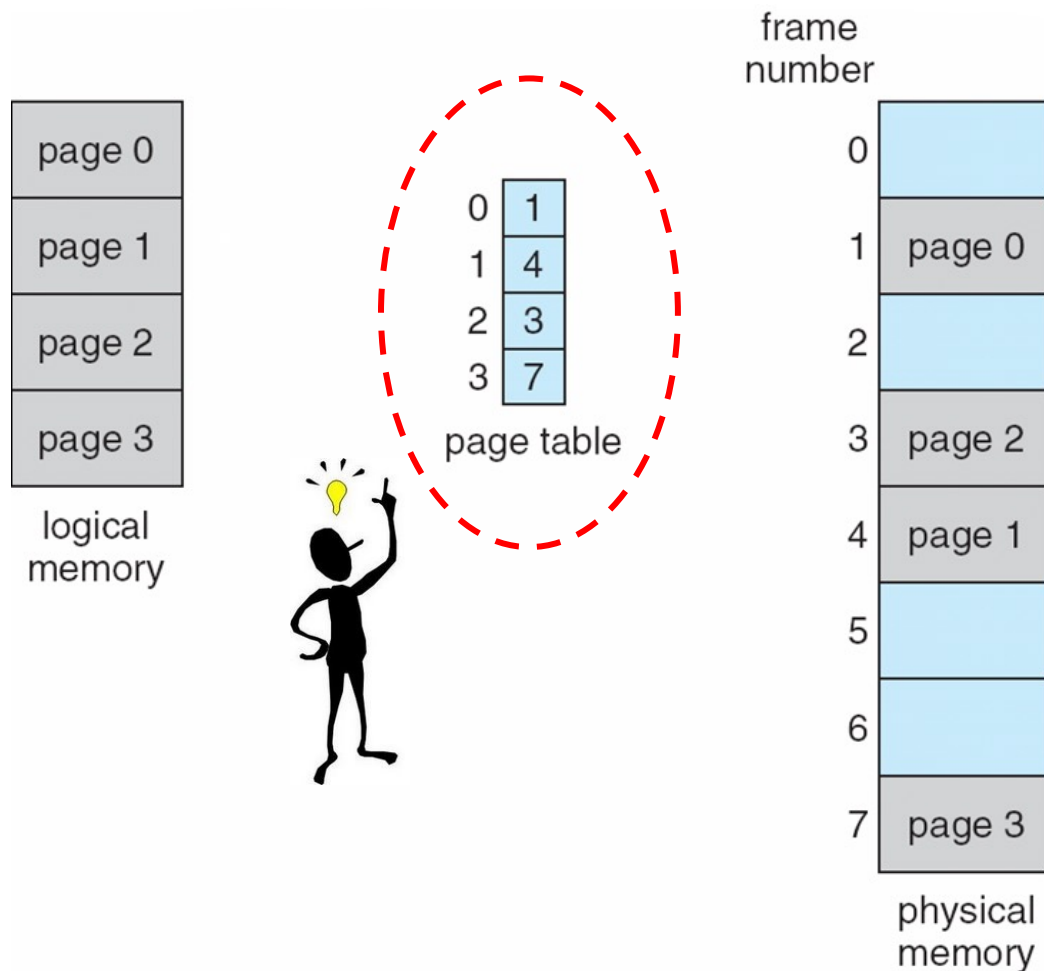
- Split logical memory in pages
- Split physical memory in frames
- Keep a program (disk) organized in pages

Page size = Frame size!



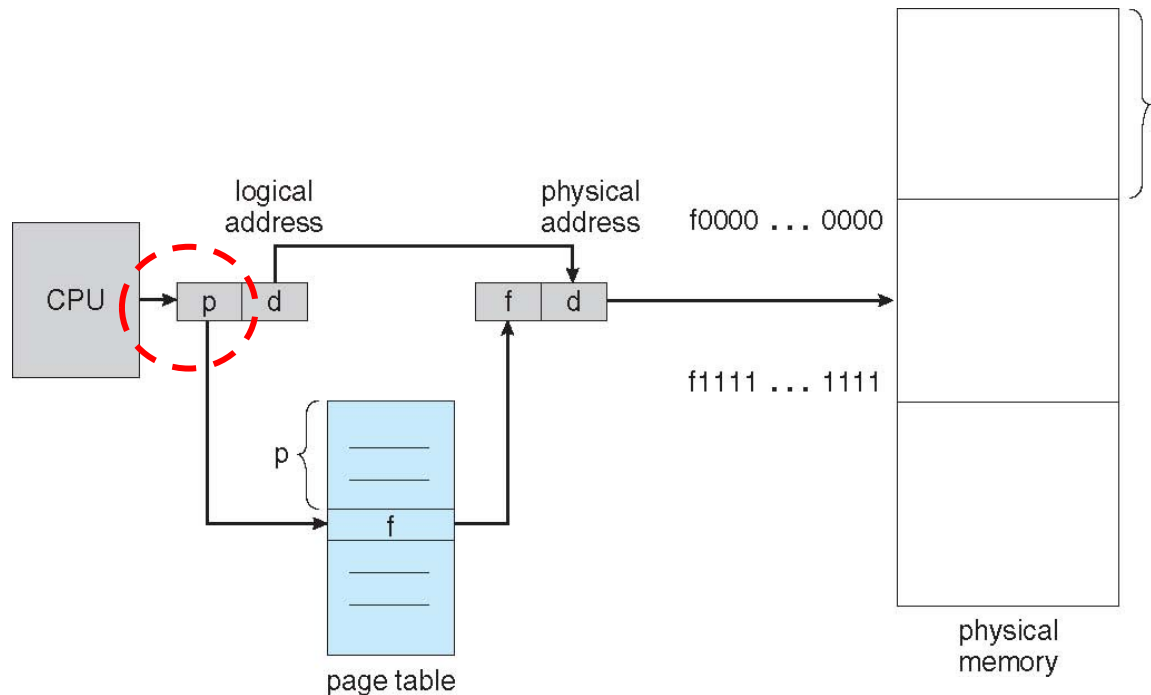
How do we keep track of where each page is?

Paging – the page table



- Page 0 → Frame 1
- Page 1 → Frame 4
- Page 2 → Frame 3
- Page 3 → Frame 7

Paging – hardware support

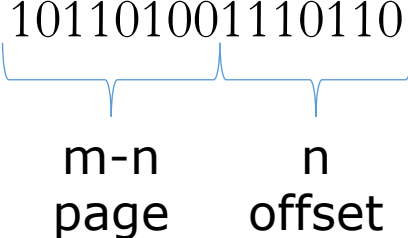


How does the CPU know which page it should access?

Similarly to Segmentation, we can access physical memory specifying page + offset

Paging – from logical address to physical address (i)

- If page/frame size is power of 2 $\rightarrow 2^n$
- If size of logical space is power of 2 $\rightarrow 2^m$

Logical address (binary) 
The diagram shows the binary address 101101001110110. A blue bracket under the first seven bits (1011010) is labeled 'm-n' and 'page'. A blue bracket under the last four bits (1110) is labeled 'n' and 'offset'.

Paging – from logical address to physical address (ii)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Page size = 2^2 bytes ($n=2$)

Size of logical space = 2^4 bytes ($m=4$)

→ logical space = 4 pages



Physical address of logical address 7?

0 1 1 1
page 1 offset 3

Frame 6 offset 3
→ $6 \times 4 + 3 = 27$

Paging - Fragmentation

- No external fragmentation
 - Process needs a page? Can take any free frame
- Internal fragmentation
 - Last frame assigned to a process' page might not be full
 - On average, half a page/frame is wasted for each process



- Small pages → less fragmentation / more overhead (e.g., page table size)
- Large pages → more fragmentation / less overhead

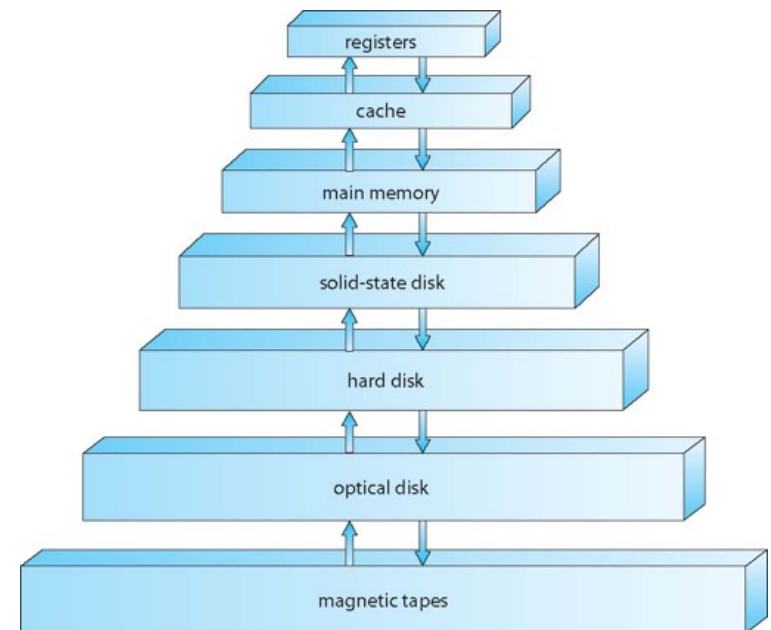
Questions

47

Vincenzo Gulisano

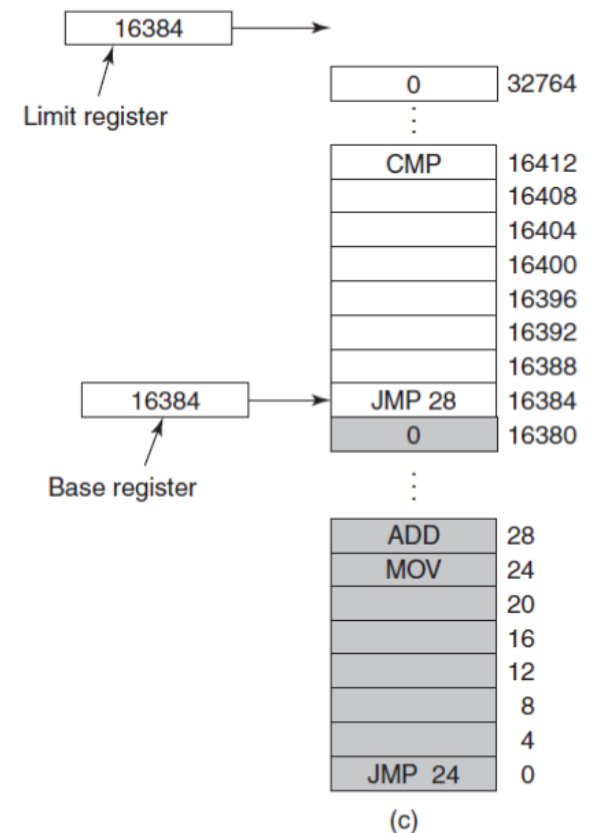
Which of these memories can be accessed directly by the CPU?

1. Registers, cache and main memory
2. Registers, cache, main memory and disk (solid-state)
3. Registers, cache, main memory and disk (any)



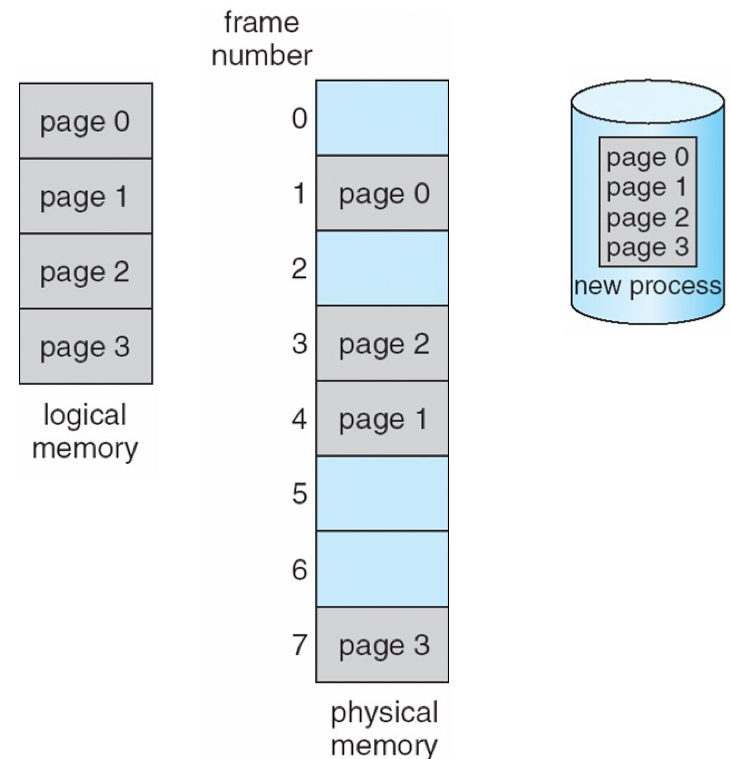
The base and limit registers in the figure contain the same value. Is it OK?

1. Yes, it is.
2. No, it is not.



Paging suffers from...

1. External fragmentation
2. Internal fragmentation
3. Does not suffer fragmentation



Process p address space is 128K. There are 16K of physical memory available (page size is 1K).

1. The page table contains 128 entries
2. The page table contains 16 entries
3. The page table contains 8 entries

Suppose a process P has size of 100 bytes. Compute (1) the number of wasted bytes caused by internal fragmentation if the page and frame sizes are set to 2^4 bytes and (2) the size of the page table (suppose each page-table entry occupies 1 byte).

Assuming a 1 KB page size, what are the page numbers and offsets for the following address references (provided as decimal numbers):

2375

19366

30000

256

16385

Consider the following segment table:

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1352	96

Tell which physical address would be accessed for the following logical addresses

0,430

1,10

2,500

3,400

4,112